Universidad Centroamericana José Simeón Cañas
Departamento de Electrónica e Informática.

Administración de bases de datos
Laboratorio 4 Ejercicios G3

# Grupo 3 - AdventureWorks - Inventario

**Base de datos a utilizar:** AdventureWorks2022
**Entorno:** contenedor Docker `sqlserver-2025` de la clase (`localhost,1434`, login `sa`)

**Indicaciones**

Restaurar la base de datos **AdventureWorks** desde su archivo `AdventureWorks2022.bak` (el instructor indicará la ruta dentro del contenedor). Trabaja con el nombre `AdventureWorks2022`.

El laboratorio consta de 4 ejercicios prácticos. Debes elegir y resolver **3**. Los ejercicios son independientes entre sí.

Entrega un archivo `.sql` y el PDF `tu-carnet_Laboratorio4.pdf` con capturas. Activa siempre el plan de ejecución real (`Ctrl + M`) y `SET STATISTICS IO ON`.

En las tablas pequeñas el optimizador puede mantener un Scan aunque exista un índice. Si te ocurre, documenta las lecturas lógicas y explica la decisión del motor.

Trabaja siempre sobre una base recién restaurada y no modifiques los datos existentes, salvo donde el ejercicio lo pida.

## Ejercicio 1 · Esquemas y secuencias

**Problema.** Bodega quiere mover las ubicaciones físicas a un esquema propio y llevar en él un registro de auditoría, cuyos identificadores los genere automáticamente una secuencia en lugar de escribirse a mano.

**Requerimiento**

1. Crear el esquema `Bodega` con propietario `dbo`.
2. Transferir la tabla existente `Production.Location` al nuevo esquema con `ALTER SCHEMA ... TRANSFER` y comprobar su nueva ubicación en el catálogo del sistema.
3. Crear la secuencia `Bodega.seq_registro_bodega` de tipo entero, que inicie en 200, avance de 1 en 1, no haga ciclo y reserve 20 valores en memoria (CACHE).
4. Crear una **nueva tabla** `Bodega.Registro` con las columnas `id_registro` (INT), `usuario` (VARCHAR(50)) y `fecha` (DATE). Configura `id_registro` para que, por defecto (DEFAULT), tome el siguiente valor de `Bodega.seq_registro_bodega`.
5. Insertar dos filas en `Bodega.Registro` indicando únicamente el usuario y la fecha, y consultar la tabla para verificar que la secuencia asignó los identificadores correctamente.

**Salida esperada.** La tabla `Location` aparece en el esquema `Bodega` y ya no en el original, y la nueva tabla `Registro` queda creada en ese mismo esquema. La consulta al catálogo de secuencias muestra `seq_registro_bodega` con valor inicial 200 e incremento 1. Las dos filas insertadas reciben automáticamente los identificadores 200 y 201 (sin que los escribas en el `INSERT`), junto con el usuario y la fecha indicados.

## Ejercicio 2 · Índices y planes de ejecución

**Problema.** Control de inventario revisa los movimientos de un solo día (11 de julio de 2014) y la consulta lee todo el historial.

```
SELECT TransactionID, ProductID, TransactionType, Quantity
FROM Production.TransactionHistory
WHERE TransactionDate >= '20140711' AND TransactionDate < '20140712';
```

**Requerimiento**

1. Ejecutar la consulta con el plan real y `SET STATISTICS IO ON`. Identificar el operador de acceso a la tabla y anotar las lecturas lógicas.
2. Crear el índice (agrupado o no agrupado) que corresponda a la columna de búsqueda `TransactionDate`. Justifica por escrito tu elección.
3. Repetir la consulta y comparar operador y lecturas lógicas.

**Salida esperada.** Antes del índice, el plan muestra un Clustered Index Scan (o Table Scan) con lecturas iguales al tamaño de la tabla. Después, el plan debería pasar a un Index Seek sobre el índice nuevo (con Key Lookup si pides columnas que el índice no contiene) y las lecturas lógicas deberían bajar de forma notable. Si el optimizador conserva el Scan, explica por qué con las filas estimadas y las lecturas.

## Ejercicio 3 · Optimizando el Key Lookup con INCLUDE

**Problema.** Costos consulta el historial de un producto concreto (por ejemplo, ProductID = 717) y necesita la fecha, la cantidad y el costo real de cada movimiento.

```
SELECT TransactionDate, Quantity, ActualCost
FROM Production.TransactionHistory
WHERE ProductID = 717;
```

**Requerimiento**

1. Ejecutar la consulta con el plan real y `SET STATISTICS IO ON`. Localizar el Key Lookup y anotar las lecturas lógicas.
2. Crear (o reemplazar el índice existente `IX_TransactionHistory_ProductID`, documentando tu decisión) un índice no agrupado que cubra la consulta: decide qué columna va como clave y cuáles como columnas incluidas (TransactionDate, Quantity y ActualCost deben quedar disponibles en el índice).
3. Repetir la consulta y comparar el plan y las lecturas lógicas con las del paso 1. Si conservas ambos índices, verifica en el plan cuál usa el optimizador.

**Salida esperada.** Al inicio, el plan muestra un Index Seek seguido de un Key Lookup unidos por Nested Loops. Con el índice cubierto desaparece el Key Lookup, queda un único Index Seek y las lecturas lógicas disminuyen. Explica en dos líneas por qué las columnas incluidas no forman parte de la clave de búsqueda.

## Ejercicio 4 · Análisis de JOINs y rendimiento (avanzado)

**Problema.** Gerencia pide el costo de los movimientos de venta del cuarto trimestre de 2013 por categoría y subcategoría de producto.

```
SELECT pc.Name AS Categoria, ps.Name AS Subcategoria,
       COUNT(*) AS Movimientos, SUM(th.ActualCost) AS Costo
FROM Production.TransactionHistory AS th
JOIN Production.Product AS p ON p.ProductID = th.ProductID
JOIN Production.ProductSubcategory AS ps ON ps.ProductSubcategoryID =
p.ProductSubcategoryID
JOIN Production.ProductCategory AS pc ON pc.ProductCategoryID = ps.ProductCategoryID
WHERE th.TransactionType = N'S'
  AND th.TransactionDate >= '20131001' AND th.TransactionDate < '20140101'
GROUP BY pc.Name, ps.Name
ORDER BY Costo DESC;
```

**Requerimiento**

1. Ejecutar la consulta con el plan real, `SET STATISTICS IO ON` y `SET STATISTICS TIME ON`.
2. Documentar las lecturas lógicas de cada tabla (TransactionHistory, Product, ProductSubcategory y ProductCategory), el tiempo de CPU y el tiempo transcurrido.
3. Identificar el algoritmo de unión que usó el motor entre cada par de tablas (Nested Loops, Hash Match o Merge Join) y el operador de mayor costo porcentual en el plan.
4. Proponer y crear **un** índice que reduzca el costo de la operación más pesada (indica nombre, columnas clave y columnas incluidas) y volver a medir.
5. Redactar una conclusión de cinco a siete líneas comparando antes y después, y explicando si cambió el algoritmo de unión.

**Salida esperada.** Capturas del plan y de la pestaña Mensajes antes y después, una lista con lecturas lógicas por tabla, CPU y tiempo transcurrido, el algoritmo de unión de cada par de tablas y el operador más costoso. Tras crear el índice, las lecturas lógicas de la tabla de mayor volumen deberían bajar. Si la mejora es pequeña, justifícalo con el tamaño de las tablas.